Graph Databases as a Psychological Method: A Tutorial

Juan C. Correa[1*] and María del Pilar García-Chitiva[2]

[1*]Departmento de Estudios Empresariales, Universidad Iberoamericana, Mexico City, Mexico, 01219.
[2]Escuela de Educación, Politécnico Grancolombiano, Bogotá, Colombia, 110231.

*Corresponding author(s). E-mail(s): juanc.correa@ibero.mx;

Abstract

This tutorial presents labeled property graphs as a psychological method. We show how this paradigm addresses tabular constraints that fragment, duplicate, or obscure relational behavioral structure. Using the Speed-Dating replication dataset as a running example, we demonstrate how graph databases preserve relational structure, accommodate asymmetric dyadic roles, and support querying of longitudinal and conditional processes. We discuss methodological implications of graph-based representations for psychological research, including structural approaches to missingness, scalability as an epistemic constraint, and ontological transparency. Together, these features position graph databases as a psychological method.

Keywords: Graph database, Network modeling, complex behavior

1 Introduction

The relational nature of psychological data is evident across multiple research domains. For example, Guimerà, Uzzi, Spiro, and Amaral (2005) examined team assembly mechanisms to show how patterns of collaboration emerge from relational constraints and Borsboom and Cramer (2013) introduced network-based models to analyze the causal interplay among psychopathological symptoms. What these approaches share is that their relational structure is treated as an analytical construction, reconstructed from rectangular data formats, rather than as a database design principle. Treating psychological data as a graph database design represents a fundamentally different methodological paradigm that has received little explicit attention in psychological research.

Relational databases dominate psychological research for good reasons. They store data in tables where columns represent variables, rows represent cases, and relationships between entities are managed through the so-called SQL or "Structured Query Language" (Soto, 2025). This paradigm works well when data are tidy and relationships are simple (Wickham, Çetinkaya Rundel, & Grolemund, 2023). However, as psychological phenomena grow more complex (i.e., spanning multiple levels of analysis, unfolding over time, and involving entities that relate to each other in qualitatively distinct ways) the tabular constraints of relational databases become a methodological liability rather than an asset.

An alternative paradigm, broadly referred to as "Not Only SQL" (NoSQL), includes systems designed for non-tabular, highly connected data. Among these, graph databases are particularly well suited for psychological research because they treat relationships as first-class elements rather than as implicit byproducts of table joins (Robinson, Webber, & Eifrem, 2015). This database paradigm allows behavior to be modeled as a complex system in a way that differs fundamentally from traditional approaches (Guastello, Koopmans, & Pincus, 2009). In this article, we examine graph databases as a methodological paradigm in which nodes and edges (rather than tables and joins) constitute the basic representational units of behavior. Understanding why this database paradigm remains largely absent from psychological research requires first acknowledging what already exists.

Computational libraries such as `sna`, `igraph`, `qgraph`, or `psychonetrics` have made network analysis accessible to researchers working entirely within `R` (Luke, 2015), and their adoption in psychology is well-known (Mair, 2018). However, these libraries mainly address the statistical analysis of relational data, leaving the management of it under the hood (i.e., the data must first be extracted from a rectangular format, reshaped into an edge list or adjacency matrix, and then passed to the analytic tool). Thus, such a workflow reconstructs relational structure at the analysis stage rather than preserving it from the moment of data collection. Graph database management systems, by contrast, store, query, and retrieve data in graph form natively, so that relational structure is not an analytical construction but a representational commitment. To our knowledge, this distinction has received no systematic treatment as a psychological method.

This article provides a step-by-step tutorial for implementing graph databases in psychological research. In particular, our tutorial revisits the Speed-Dating replication dataset (Selterman, Chagnon, & Mackinnon, 2015, 2023), whose dyadic, longitudinal, and multimodal structure provides an ideal scenario for demonstrating how graph databases preserve relational interactions, handle structural missingness without numerical encoding, and support queries that rectangular formats cannot express without costly joins or data duplication.

The remainder of this article is organized as follows. Section 2 introduces the conceptual foundations of graph databases, contrasting their representational logic with that of relational databases and defining the core elements — nodes, edges, labels, and properties — that make them particularly well-suited for psychological data. Section 3 introduces important epistemic considerations of graph databases in psychological research. Section 4 presents the step-by-step tutorial itself through the Speed-Dating replication dataset (Selterman et al., 2023). This dataset shows how its dyadic, longitudinal, and multimodal structure exposes the limitations of rectangular formats. Section 5 discusses the broader methodological implications of graph databases for psychological research, with particular attention to structural missingness, scalability, and ontological transparency. Section 6 concludes with a discussion of limitations and directions for future work. All code, data, and a fully reproducible RMarkdown file are available via the Open Science Framework at [OSF link].

2 Graph Databases and Labeled Property Graphs

Graph databases are data management systems that implement the basic operations of **C**reate, **R**ead, **U**pdate, and **D**elete (CRUD) on a graph data model, in which entities are represented as nodes and relationships are represented as edges (Robinson et al., 2015). A fundamental element of this paradigm is the labeled property graph (LPG), in which both nodes and edges can carry labels and properties, allowing attributes of entities and relationships to be stored directly within the graph structure. Unlike relational databases (i.e., where relationships are encoded indirectly through foreign keys and join operations) graph databases treat relationships as first-class components of the data model. As a result, relational structure becomes an explicit feature of data representation, rather than a construct introduced retrospectively at the analytical stage of research. To make these abstract representational principles concrete, we now illustrate them using a well-known relational dataset in psychology.

The graph database paradigm fits naturally onto relational phenomena such as the exhibited differences between men and women when dating and looking for a romantic partner (Selterman et al., 2015). We decide to revisit this phenomenon through the Speed-Dating dataset (Selterman et al., 2023). This dataset has the records of participants who went on brief, 4-minute "dates" with potential romantic partners. After attending the speed-dating event, participants were given the opportunity to select people they "matched" with and contact each other as they see fit. Selterman et al. (2023) also assessed participants' degree of attraction and relationship initiation with follow up questions that were dependent on whether or not participants had subsequent interactions with mutual matches (e.g., "What is the current status of your relationship with [name]?" (a) dating seriously, (b) dating casually, etc.). These "pivot" questions were administered only if the participant and match "hung out" again, engaged in romantic behavior including dates and/or labeled the match as a romantic partner (or having romantic potential), engaged in sexual activity, etc. The sample consisted of 307 people (49.8% female), and each person had average of 9.28 (SD = 2.14) speed dates. Only 141 (45.9%) participants completed the age and ethnicity demographic variables. These participants were an average of 18.99 years old (SD = 2.86). The sample primarily identified as white (62.4%), Hispanic (9.9%), African American (10.6%), Asian/Pacific Islander (11.3%), or "other" (5.7%). Figure 1 shows a labeled property graph for the Speed-Dating dataset (Selterman et al., 2023).

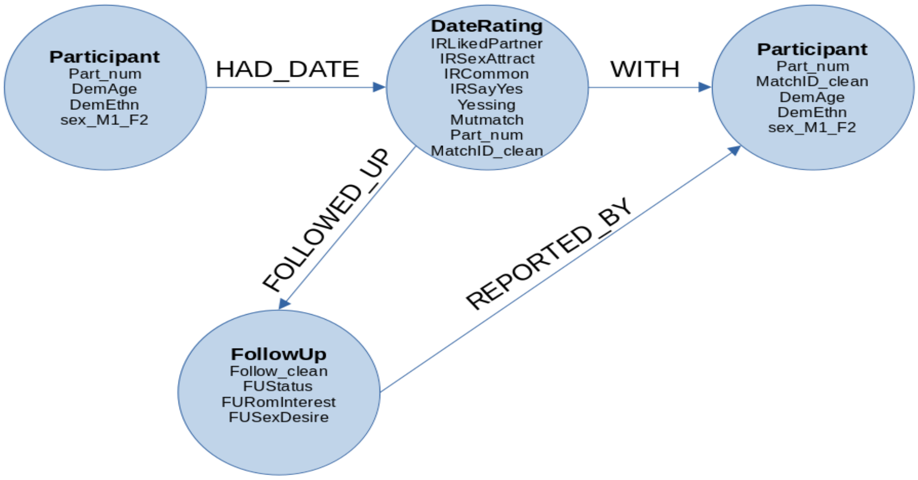

Fig. 1 A labeled property graph of the speed-dating dataset by Selterman et al. (2023)

2.1 Entities as Nodes

Figure 1 shows how participants interacted in the speed-dating design. Nodes (in light blue) have their own identification (in bold letters) and properties (in regular letters). Nodes connect through labeled directed links (i.e., `HAD_DATE`, `WITH`, `FOLLOWED_UP`, and `REPORTED_BY`). A subtle distinction is visible between the four properties associated with the `Participant` node to the left of the `DateRating` node and the five properties associated with the `Participant` node to the right. As the speed-dating design implied that men and women rated each other, the role of "rater" (Part_num) and "ratee" (MatchID_clean) should be explicitly differentiated. Such a distinction is evident in the dataset: the first row shows that participant 1 (male) matched with participant 116 (female). Participant 1 evaluated Participant 116 with a score of 9 in IRLikedParter, while row 361 shows participant 116 evaluated participant 1 with a score of 6 in IRLikedParner.

The `DateRating` node encompasses a series of dyadic interactions and store the full set of impression-formation ratings provided by both participants (Part_num, and MatchID_clean) during the speed-dating (e.g., IRLikedPartner, IRSexAttract, IRCommon, IRSayYes, Yessing, Mutmatch). The `FollowUp` node represents week-level longitudinal responses, including relationship status, romantic interest, sexual desire, and commitment indicators (e.g., FUStatus, FURomInterest, FUSexDesire, FUCommitted), captured only for participants who matched and continued interacting.

2.2 Relationships as Edges

Edges define how these nodes are linked and encode psychological structure directly. The `HAD_DATE` edge connect participants to the `DateRating` in which they took part, indicating the rater's perspective on that interaction. The edge labeled as `WITH` connects the `DateRating` back to the other participant, allowing the system to explicitly represent dyadic structure. The edge labeled as `FOLLOWED_UP` links `Participants` to their `FollowUp` node, representing relational trajectories across weeks. Finally, the `Participant`node to the right of the `DateRating` node receives the connection `REPORTED_BY` from the `FollowUp`node. This connection mirrors the fact that both participants (Part_num, and MatchID_clean) had to report their subjective assessments on the romantic parter they matched with during follow-up sessions.

2.3 Why Graph Databases Matter for Psychological Research

The structure of Figure 1 offers several methodological advantages. Psychological data retain their natural relational structure. Dyadic interactions are modeled as shared events rather than duplicated rows, preserving the topology of social and romantic processes as they occur in the real world. Temporal processes become navigable. The `FollowUp` node can be traversed sequentially, enabling queries such as "How early do committed feelings emerge among mutually matched pairs?' ' or "Does initial sexual attraction predict later romantic interest?' '

In addition, missingness is handled structurally, not numerically. Beyond missing values within rows, graph databases also recover entities that are entirely absent from the rectangular file. In the Speed-Dating dataset, 87 participants appear exclusively as targets of others' ratings (MatchID_clean) but have no rows of their own as raters (Part_num). A relational table cannot represent these individuals without an additional join table; the graph instantiates them as nodes automatically, preserving the full social structure of the speed-dating event. Participants who did not match simply lack `FollowUp`; no NA encoding is required. The model is inherently extensible. Additional constructs, such as personality traits or behavioral logs, can be attached as new nodes or properties without reconfiguring the existing database schema. In sum, LPGs allow behavioral scientists to represent complex, multi-level psychological phenomena in a way that relational tables cannot. The Speed-Dating schema illustrated in Figure 1 makes clear how impression formation, dyadic reciprocity, and longitudinal relationship development can be captured and analyzed within a single unified model.

3 Graph Databases: Epistemic Considerations

In this section, we focus not on implementation, but on the epistemic considerations of adopting graph databases in psychological research. Graph databases and their labeled property graph paradigm provide an infrastructure that enables the systematic incorporation of complex systems thinking into psychological inquiry. In psychology and the behavioral sciences, complex systems thinking is not novel (Guastello et al., 2009; Luce, 1999), although more recent work has emphasized its interdisciplinary potential (Krakauer, 2019). This emphasis aligns with recent reviews that propose methodological innovations as a means of strengthening intra- and interdisciplinary integration across basic and applied domains of behavior analysis (Elcoro, Diller, & Correa, 2023).

As a psychological method, the graph database paradigm shares some similarities with the *nomological nets* originally proposed by (Cronbach & Meehl, 1955). Both emphasize relational structure and the systematic articulation of theoretical commitments. However, labeled property graph models constitute a more general-purpose representational framework that transcends the scope of classical nomological nets (see Table 1).

Table 1: Knowledge graphs and nomological nets

Feature	Labeled Property Graphs	Nomological nets
Primary field	Computer science / AI / Data engineering	Psychology / Philosophy of Science
Primary goal	Data integration, search, automated reasoning	Establishing construct validity (i.e., theory testing)
Nature of nodes	Concrete entities (e.g., “Lee J. Cronbach”, “University of Illinois”)	Theoretical constructs (e.g., “introversion”, “intelligence”)
Nature of edges	Predicates or properties (e.g., “works_at”, “collaborates_with”)	Theoretical laws or empirical correlations
Verification	Data consistency and retrieval accuracy	Statistical evidence (e.g., correlations, p-values)
Scale	Potentially billions of facts	Typically small, theory-specific

What distinguishes graph databases from analytic tools is that they constrain and enable psychological inquiry at the level of *representation*. Whereas analytic methods operate on data that have already been structured, graph databases determine how behavioral phenomena are encoded from the outset, thereby delimiting which relations, processes, and queries are epistemically viable.

This representational commitment has direct methodological consequences, particularly for research involving complex, relational, and large-scale psychological data. One such consequence concerns scalability. Because graph databases organize information around localized connections among nodes and edges, they maintain relatively stable query performance even in datasets comprising millions of entities and relationships. From a traditional perspective within psychology, working at this scale may appear implausible. However, large-scale behavioral data are now firmly within methodological consideration, as reflected in recent treatments of big data as a psychological method (Vezzoli & Zogmaister, 2023). Prior work in network science illustrates this point clearly. For example, studies of urban mobility have examined individual-level behavioral trajectories by tracking the movements of hundreds of thousands of drivers at fine temporal resolutions (Gonzalez, Hidalgo, & Barabasi, 2008). In such contexts, graph databases are particularly well suited because queries operate on localized subgraphs rather than requiring exhaustive scans of entire datasets. Performance, therefore, is not merely a technical concern but a condition for the epistemic viability of certain classes of psychological questions.

A second epistemically consequential feature of graph databases is their flexibility. Behavioral scientists routinely connect data in ways that evolve alongside theoretical commitments, empirical discoveries, and domain expertise. For this reason, graph databases often serve as the underlying infrastructure for constructing *knowledge graphs* (Barrasa & Webber, 2023). Unlike rigid schemas defined in advance, knowledge graphs allow data models to evolve in tandem with scientific understanding—an especially important feature in domains characterized by limited theory, active replication efforts, or rapid evidence accumulation (Burgos, 2025). Because graph structures are inherently additive, new nodes, relationships, labels, and subgraphs can be introduced without invalidating existing representations. From a methodological perspective, such modifications should be understood not as ad hoc adjustments, but as legitimate indicators of scientific progress under conditions of incomplete knowledge.

These representational choices bring ontological and epistemological considerations to the foreground. Ontologically, graph databases require researchers to specify explicitly which entities exist within the model, how they relate to one another, and which transformations of those entities are permissible. Epistemologically, they invite reflection on what counts as evidence, which queries are meaningful, and how explanations are warranted given the structure of the graph. In this sense, graph databases make explicit assumptions that often remain implicit in rectangular data workflows, echoing recent calls in theoretical and philosophical psychology to treat ontology and epistemology as integral components of methodological design (Burgos, 2025).

These considerations are particularly salient in contemporary psychological research that relies on automated data collection techniques. Mobile applications, such as ecological momentary assessment tools, prompt participants to report emotions or behaviors repeatedly throughout the day, while motion sensors and radio-frequency identification tags continuously record movement and location in naturalistic environments (Bak et al., 2021). In such technology-mediated inquiries, datasets tend to grow rapidly while becoming increasingly heterogeneous. Although Soto (2025) strongly advocates for relational databases in behavioral research, he also identifies several practical challenges to their adoption, including infrastructural requirements and specialized technical expertise. From the perspective advanced here, these challenges highlight the benefits of alternative data representation paradigms—such as graph databases—which align representational structure more closely with the relational nature of psychological phenomena.

In sum, labeled property graphs are not merely a technical alternative to relational schemas; they represent a methodological advance that aligns with psychology's need for flexible, scalable, and conceptually transparent data models.

4 Applying graph database in behavioral research

This section provides a step-by-step tutorial demonstrating how to (a) design a graph schema for psychological data, (b) prepare the raw dataset for import, (c) import the data into a labeled property graph using Cypher, and (d) verify that the resulting graph matches the intended schema. Readers with no prior experience with Neo4j or graph databases should be able to follow each step in sequence using only the materials described below.

4.1 STEP 1: Pre-requisites and materials

This tutorial uses the Speed-Dating replication dataset (Selterman et al., 2023), available in the Open Science Framework repository. From the Files preview menu, download speed.dating.2.zip from the Final Report option. The zip archive contains the raw data in SPSS format (.sav). This file can be opened in SPSS, PSPP, JASP, jamovi, or RStudio. The tutorial uses RStudio for all pre-processing steps.

Two software environments are required throughout the tutorial. RStudio is used to convert the raw dataset from SPSS format to a CSV file suitable for Neo4j. Neo4j Desktop is used to create the graph database instance, import the CSV, and run Cypher queries (Barrasa & Webber, 2023). Neo4j Desktop can be downloaded free of charge from its official website. The reader may consult Van Bruggen (2014) for installation details across Mac, Windows, and Linux environments. Two R packages are required: `haven` (Wickham, Miller, & Smith, 2025) for reading the SPSS file, and `utils` (R Core Team, 2025), which is included in every R installation by default and provides the write.csv() function. No additional R packages are needed for the tutorial.

4.2 STEP 2: Instance creation

Open Neo4j Desktop and click the Create Instance button in the upper right corner. Name the instance SpeedDating and set a password that is easy to recall across sessions. Once created, click Start instance. The label RUNNING in green letters should appear below the instance name, confirming that the database is active and ready to receive data. Before importing any data, the CSV file must be placed in a specific folder that Neo4j can access. With the SpeedDating instance running, click the folder icon in Neo4j Desktop to reveal the instance directory. Navigate to the import/ subfolder inside that directory. This is the only folder from which Neo4j can load CSV files using the LOAD CSV command. All import operations in this tutorial assume that the CSV file is located at this path (Figure 2).

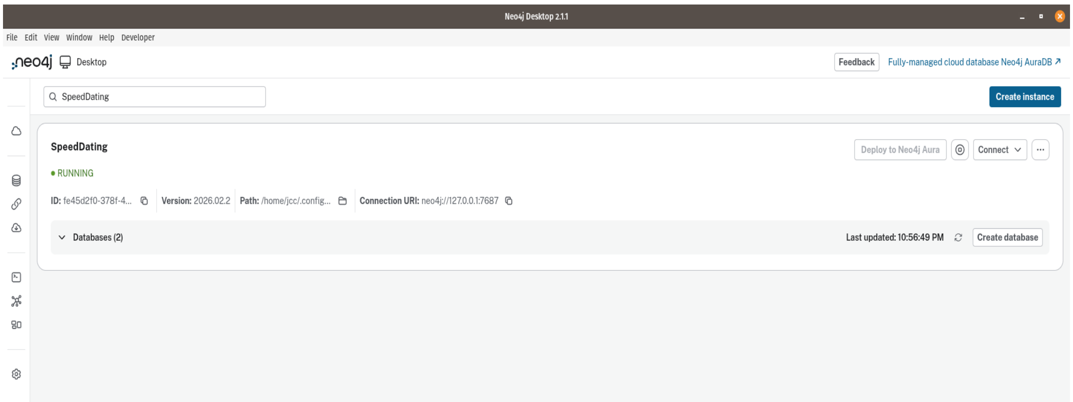

Fig. 2 The SpeedDating instance running in Neo4j

4.3 STEP 3: Designing the Graph Schema

Before writing any code, it is useful to decide how the data will be represented as a graph. This decision is what the graph database literature calls schema design, and it is the most consequential methodological step in the tutorial. A poor schema design produces a graph that is technically valid but analytically misleading; a good schema design makes the psychological structure of the data immediately visible.

The speed-dating dataset has a three-level structure. At the highest level are individual participants. At the intermediate level are the dyadic interactions that took place during the speed-dating event, each representing the subjective impressions that one participant formed of another. At the lowest level are the longitudinal follow-up assessments, collected weekly over ten weeks for pairs who continued interacting after the event. This three-level structure maps directly onto three node types. A key design decision concerns the intermediate level. An obvious but incorrect approach would be to model a speed-dating interaction as a single shared node connecting two participants symmetrically. The data do not support this interpretation. Each participant rated every partner independently, and those ratings are asymmetric: participant 1 rated participant 116 with a score of 9 on IRLikedPartner, while participant 116 rated participant 1 with a score of 6 on the same item. These are two distinct perceptual events, not one shared event. The intermediate node is therefore called DateRating rather than DateEvent, and each DateRating node represents the impressions formed by one specific rater about one specific partner.

Figure 1 illustrates the resulting labeled property graph. It contains three node types and four directed relationships. Table 2 summarizes each element of the schema.

Table 2: Schema elements of the Speed-Dating labeled property graph.

Element	Type	Description
Participant	Node	One node per individual. Carries Part_num (rater side) or MatchID_clean (partner side), DemAge, DemEthn, and sex_M1_F2.
DateRating	Node	One node per rater-partner pair. Carries Part_num, MatchID_clean, IRLikedPartner, IRSexAttract, IRCommon, IRSayYes, Yessing, and Mutmatch. Uniquely identified by the composite key Part_num + MatchID_clean.

Element	Type	Description
FollowUp	Node	One node per rater-partner-week triplet, created only when at least one follow-up variable is non-missing. Carries Follow_clean, FUStatus, FURomInterest, FUSexDesire, and FUCommitted.
HAD_DATE	Edge	Directed edge from the rater Participant to their DateRating. Encodes who formed the impressions.
WITH	Edge	Directed edge from DateRating to the partner Participant. Encodes who was evaluated.
FOLLOWED_UP	Edge	Directed edge from DateRating to FollowUp. Links an impression to the longitudinal assessments that followed it.
REPORTED_BY	Edge	Directed edge from FollowUp to the partner Participant (dashed in Figure 1). Makes explicit that follow-up reports are subjective assessments directed at a specific partner.

Two properties deserve special attention because they do not appear as columns in the original CSV file. The composite identifier of each `DateRating` node is constructed by concatenating Part_num and MatchID_clean into a single string of the form "Part_num_MatchID_clean" (e.g., "1_116" for participant 1 rating participant 116). This composite key is necessary because neither column alone uniquely identifies a dyadic perception: the same MatchID_clean value appears in the rows of many different raters. Similarly, the `FollowUp` node is identified by the combination of the composite rater-partner key and Follow_clean, which records the week number (1 through 10). Both composite identifiers are constructed during the import step and do not require any modification of the original CSV file. This design decision has an empirically verifiable consequence. In the Speed-Dating dataset, Part_num contains 309 unique values while MatchID_clean contains 378. Of these, 291 values appear in both columns, meaning that 87 individuals were rated by others during the speed-dating event but have no rows of their own in the rectangular file — they exist only as targets of someone else's perception. In a relational table, these 87 participants are structurally invisible unless a separate lookup table is created. In the graph, they emerge automatically as Participant nodes during import, producing the 396 nodes confirmed in Step 6. Their presence in the graph without a corresponding presence in the rectangular file illustrates what we mean by schema flexibility: the graph preserves entities that the tabular format omits by design.

4.4 STEP 4: Data preparation in R

The original dataset is distributed as an SPSS file (.sav). Neo4j does not read SPSS files directly, so the first task is to convert it to CSV format using RStudio. The following R script reads the .sav file, replaces all missing values with empty strings (which is the format that Neo4j's LOAD CSV command expects), and writes the result to disk as speed_dating.csv.

```
# Install the haven package if not already installed
# install.packages("haven")
library(haven)
# Read the SPSS file
speed_dating <- read_sav("speed.dating.3lev.small.july25.2014.sav")
# Convert labelled columns to plain numeric vectors
speed_dating <- as.data.frame(lapply(speed_dating, function(x) {
  if (inherits(x, "haven_labelled")) as.numeric(x) else x
}))
# Write to CSV without row numbers
# na = "" ensures missing values appear as empty cells,
# which is the format Neo4j expects for LOAD CSV
write.csv(
  speed_dating,
  "speed_dating.csv",
  row.names = FALSE,
  na = ""
)
```

Two details in this script are critical for a successful import. First, row.names = `FALSE` prevents R from adding an unnamed index column at the start of the file. Neo4j rejects CSV files that contain empty column headers, so omitting the row index is mandatory. Second, `na = ""` replaces R's default `NA` token with an empty string. The Cypher import script uses trim(row.column) <> “” to detect missing values, and this check only works correctly when missing cells are genuinely empty rather than containing the literal text `NA`. Note that once the script has run, copy speed_dating.csv into the import/ folder of the SpeedDating Neo4j instance before proceeding to Step 5. The location of this folder was described in Step 2.

4.5 STEP 5: Data Import into Neo4j

With the CSV file in the import/ folder, open Neo4j Browser by clicking the Connect button in Neo4j Desktop and then selecting Open. The Cypher import script below must be executed in six sequential blocks. Each block is a self-contained unit; paste and run one block at a time rather than submitting the entire script at once. If an error occurs in any block, it can be corrected and re-run without affecting the blocks that have already completed successfully.

- Block 1 — Constraints and indexes. This block defines the uniqueness constraints that prevent duplicate nodes from being created during import. Constraints must exist before any nodes are loaded. Execute this block first.

```
:param {
  file_path_root: 'file:///',
  file_0: 'speed_dating.csv'
};

CREATE CONSTRAINT `participant_id_unique` IF NOT EXISTS
  FOR (p:Participant) REQUIRE p.Part_num IS UNIQUE;

CREATE CONSTRAINT `date_rating_id_unique` IF NOT EXISTS
  FOR (d:DateRating) REQUIRE d.date_rating_id IS UNIQUE;

CREATE INDEX `followup_lookup` IF NOT EXISTS
  FOR (f:FollowUp) ON (f.follow_id);
```

- Block 2 — Participant nodes. One node is created for each unique value of Part_num. Because DemAge and DemEthn are missing for approximately 55% of participants, the ON MATCH SET clause fills in those properties only when a non-empty value is encountered, avoiding overwriting a previously recorded value with a missing one.

```
LOAD CSV WITH HEADERS FROM ($file_path_root + $file_0) AS row
WITH row WHERE NOT toInteger(trim(row.Part_num)) IS NULL
CALL (row) {
  MERGE (p:Participant {Part_num: toInteger(trim(row.Part_num))})
  ON CREATE SET
    p.sex_M1_F2 = toFloat(trim(row.sex_M1_F2)),
    p.DemAge    = CASE WHEN trim(row.DemAge) <> ''
                       THEN toFloat(trim(row.DemAge)) ELSE null END,
    p.DemEthn   = CASE WHEN trim(row.DemEthn) <> ''
                       THEN toInteger(trim(row.DemEthn)) ELSE null END
  ON MATCH SET
    p.DemAge    = CASE WHEN p.DemAge  IS NULL AND trim(row.DemAge)  <> ''
                       THEN toFloat(trim(row.DemAge)) ELSE p.DemAge END,
    p.DemEthn   = CASE WHEN p.DemEthn IS NULL AND trim(row.DemEthn) <> ''
                       THEN toInteger(trim(row.DemEthn)) ELSE p.DemEthn END
} IN TRANSACTIONS OF 10000 ROWS;
```

Note that MatchID_clean is the Part_num of the partner. Both columns draw from the same population of participants, so all participants are stored under the single property Part_num regardless of the role they played in a given interaction.

- Block 3 — The DateRating node. This node represents each unique rater-partner pair. Because the IR variables were measured once at the speed-dating event but are repeated across all ten follow-up rows in the CSV, the WHERE clause restricts loading to Follow_clean = 1, which contains the original event-level ratings. The composite identifier date_rating_id concatenates Part_num and MatchID_clean and is used in subsequent blocks to match these nodes without ambiguity.

```
LOAD CSV WITH HEADERS FROM ($file_path_root + $file_0) AS row
```

```
WITH row WHERE toInteger(trim(row.Follow_clean)) = 1
CALL (row) {
  MERGE (d:DateRating {
    date_rating_id: toString(toInteger(trim(row.Part_num)))
                  + '_'
                  + toString(toInteger(trim(row.MatchID_clean)))
  })
  ON CREATE SET
    d.Part_num        = toInteger(trim(row.Part_num)),
    d.MatchID_clean   = toInteger(trim(row.MatchID_clean)),
    d.IRLikedPartner  = CASE WHEN trim(row.IRLikedPartner) <> ''
                             THEN toFloat(trim(row.IRLikedPartner)) ELSE null END,
    d.IRSexAttract    = CASE WHEN trim(row.IRSexAttract)   <> ''
                             THEN toFloat(trim(row.IRSexAttract))   ELSE null END,
    d.IRCommon        = CASE WHEN trim(row.IRCommon)       <> ''
                             THEN toFloat(trim(row.IRCommon))       ELSE null END,
    d.IRSayYes        = CASE WHEN trim(row.IRSayYes)       <> ''
                             THEN toFloat(trim(row.IRSayYes))       ELSE null END,
    d.Yessing         = CASE WHEN trim(row.yessing)        <> ''
                             THEN toInteger(trim(row.yessing))      ELSE null END,
    d.Mutmatch        = CASE WHEN trim(row.mutmatch)       <> ''
                             THEN toInteger(trim(row.mutmatch))     ELSE null END
} IN TRANSACTIONS OF 10000 ROWS;
```

- Block 4 — The FollowUp node. This node represents the rater-partner-week triplet, but only for rows where at least one follow-up variable contains a non-missing value. This prevents the graph from being inflated by approximately 26,700 structurally empty nodes that correspond to pairs who did not interact after the speed-dating event. The absence of a FollowUp node is itself informative as it indicates that no follow-up interaction occurred, without requiring any numerical encoding such as NA.

```
LOAD CSV WITH HEADERS FROM ($file_path_root + $file_0) AS row
WITH row
WHERE (
  trim(row.FURomInterest) <> '' OR trim(row.FUSexDesire)  <> '' OR
  trim(row.FUCommitted)   <> '' OR trim(row.FUStatus)     <> '' OR
  trim(row.FUEager)       <> ''
)
CALL (row) {
  MERGE (f:FollowUp {
    follow_id: toString(toInteger(trim(row.Part_num)))
             + '_'
             + toString(toInteger(trim(row.MatchID_clean)))
             + '_'
             + toString(toInteger(trim(row.Follow_clean)))
```

```
  })
  ON CREATE SET
    f.Part_num      = toInteger(trim(row.Part_num)),
    f.MatchID_clean = toInteger(trim(row.MatchID_clean)),
    f.Follow_clean  = toInteger(trim(row.Follow_clean)),
    f.FUStatus       = CASE WHEN trim(row.FUStatus)      <> ''
                            THEN toFloat(trim(row.FUStatus))      ELSE null END,
    f.FURomInterest  = CASE WHEN trim(row.FURomInterest) <> ''
                            THEN toFloat(trim(row.FURomInterest)) ELSE null END,
    f.FUSexDesire    = CASE WHEN trim(row.FUSexDesire)   <> ''
                            THEN toFloat(trim(row.FUSexDesire))   ELSE null END,
    f.FUCommitted    = CASE WHEN trim(row.FUCommitted)   <> ''
                            THEN toFloat(trim(row.FUCommitted))   ELSE null END,
    f.FUEager        = CASE WHEN trim(row.FUEager)       <> ''
                            THEN toFloat(trim(row.FUEager))       ELSE null END
} IN TRANSACTIONS OF 10000 ROWS;
```

- Block 5 — Relationships. This block creates the four directed relationships shown in Figure 1. Each MATCH statement locates previously created nodes using the same identifiers defined in Blocks 2 through 4; MERGE then creates the relationship only if it does not already exist.

```
// HAD_DATE: Participant (rater) --> DateRating
LOAD CSV WITH HEADERS FROM ($file_path_root + $file_0) AS row
WITH row WHERE toInteger(trim(row.Follow_clean)) = 1
CALL (row) {
  MATCH (p:Participant {Part_num: toInteger(trim(row.Part_num))})
  MATCH (d:DateRating  {date_rating_id:
    toString(toInteger(trim(row.Part_num))) + '_' +
    toString(toInteger(trim(row.MatchID_clean)))})
  MERGE (p)-[:HAD_DATE]->(d)
} IN TRANSACTIONS OF 10000 ROWS;

// WITH: DateRating --> Participant (partner)
LOAD CSV WITH HEADERS FROM ($file_path_root + $file_0) AS row
WITH row WHERE toInteger(trim(row.Follow_clean)) = 1
CALL (row) {
  MATCH (d:DateRating {date_rating_id:
    toString(toInteger(trim(row.Part_num))) + '_' +
    toString(toInteger(trim(row.MatchID_clean)))})
  MATCH (partner:Participant {Part_num: toInteger(trim(row.MatchID_clean))})
  MERGE (d)-[:WITH]->(partner)
} IN TRANSACTIONS OF 10000 ROWS;

// FOLLOWED_UP: DateRating --> FollowUp
LOAD CSV WITH HEADERS FROM ($file_path_root + $file_0) AS row
WITH row
```

```
WHERE (
  trim(row.FURomInterest) <> '' OR trim(row.FUSexDesire)  <> '' OR
  trim(row.FUCommitted)   <> '' OR trim(row.FUStatus)     <> '' OR
  trim(row.FUEager)       <> ''
)
CALL (row) {
  MATCH (d:DateRating {date_rating_id:
    toString(toInteger(trim(row.Part_num))) + '_' +
    toString(toInteger(trim(row.MatchID_clean)))})
  MATCH (f:FollowUp {follow_id:
    toString(toInteger(trim(row.Part_num))) + '_' +
    toString(toInteger(trim(row.MatchID_clean))) + '_' +
    toString(toInteger(trim(row.Follow_clean)))})
  MERGE (d)-[:FOLLOWED_UP {week: toInteger(trim(row.Follow_clean))}]->(f)
} IN TRANSACTIONS OF 10000 ROWS;

// REPORTED_BY: FollowUp --> Participant (partner)
LOAD CSV WITH HEADERS FROM ($file_path_root + $file_0) AS row
WITH row
WHERE (
  trim(row.FURomInterest) <> '' OR trim(row.FUSexDesire)  <> '' OR
  trim(row.FUCommitted)   <> '' OR trim(row.FUStatus)     <> '' OR
  trim(row.FUEager)       <> ''
)
CALL (row) {
  MATCH (f:FollowUp      {follow_id:
    toString(toInteger(trim(row.Part_num))) + '_' +
    toString(toInteger(trim(row.MatchID_clean))) + '_' +
    toString(toInteger(trim(row.Follow_clean)))})
  MATCH (partner:Participant {Part_num: toInteger(trim(row.MatchID_clean))})
  MERGE (f)-[:REPORTED_BY]->(partner)
} IN TRANSACTIONS OF 10000 ROWS;
```

- Block 6. Verification. After all five blocks have run without errors, the following queries confirm that the graph matches the intended schema. Each query should be run separately in Neo4j Browser. The expected output for each query is shown in Table 3.

```
// Query 1: Node counts by label
MATCH (n)
RETURN labels(n)[0] AS label, count(n) AS total
ORDER BY label;

// Query 2: Relationship counts by type
MATCH ()-[r]->()
RETURN type(r) AS relationship, count(r) AS total
```

```
ORDER BY relationship;

// Query 3: Confirm asymmetric ratings for the example pair in Section 2.1
// Participant 1 rated Participant 116 → expect IRLikedPartner = 9
MATCH (p:Participant {Part_num: 1})-[:HAD_DATE]->(d:DateRating)-[:WITH]
      ->(partner:Participant {Part_num: 116})
RETURN p.Part_num AS rater, partner.Part_num AS partner,
       d.IRLikedPartner AS score;

// Query 4: Visualize mutual-match dyads with its follow-up trajectory
MATCH (p:Participant)-[r1:HAD_DATE]->(d:DateRating {Mutmatch: 1})
      -[r2:FOLLOWED_UP]->(f:FollowUp)
MATCH (d)-[r3:WITH]->(partner:Participant)
RETURN p, r1, d, r2, f, r3, partner
LIMIT 40;
```

The result of the last query in block 6 should yield a subgraph like the one illustrated in Figure 3.

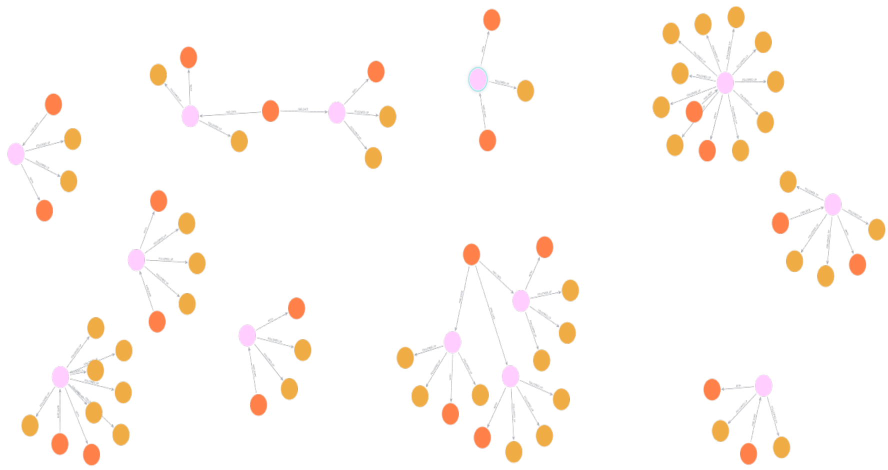

Fig. 3 Resulting subgraph showing a sample of dyads with mutual match and follow-ups

5 Methodological Implications for Psychological Research

The Speed-Dating tutorial presented in Section 4 illustrates three methodological advantages of labeled property graphs that extend beyond the specific dataset used here. This section develops each advantage as a general principle, situates it within the broader context of psychological research methods, and identifies its implications for future work.

5.1 Ontological Transparency and the Visibility of Perceptual Asymmetry

A persistent challenge in dyadic research is that the asymmetric and subjective nature of interpersonal perception is difficult to preserve in rectangular data formats. In a tabular structure, the fact that participant 1's rating of participant 116 and participant 116's rating of participant 1 are two distinct perceptual events —not two measurements of a shared reality— is not encoded in the data structure itself. It must be reconstructed at the analysis stage through careful variable naming, documentation, and researcher memory. The labeled property graph makes this asymmetry structurally explicit: each DateRating node is directed from a specific rater to a specific partner via HAD_DATE and WITH edges, and the directions of those edges are not interchangeable.

This structural commitment has a direct empirical consequence that is verifiable within the graph. Among the 84 mutual-match dyads in the Speed-Dating dataset for which both perspectives were recoverable, 70.2% showed non-zero asymmetry in IRLikedPartner —even among pairs who had explicitly chosen each other. The mean absolute difference in liking ratings within mutual-match dyads was 1.19 points on a 9-point scale (SD = 1.29), and 27.4% of dyads showed a difference of two or more points. These figures are not incidental: they constitute empirical evidence that the construct the graph represents as a directed relationship (subjective liking) behaves as a directed relationship in the data.

The Cypher query that recovers this asymmetry traverses the same two-hop path that the schema encodes:

```
// Query A: Perceptual asymmetry in mutual-match dyads
MATCH (a:Participant)-[:HAD_DATE]->(d1:DateRating {Mutmatch: 1})-[:WITH]->(b:Participant),
      (b)-[:HAD_DATE]->(d2:DateRating {Mutmatch: 1})-[:WITH]->(a)
WHERE a.Part_num < b.Part_num
  AND d1.IRLikedPartner IS NOT NULL
  AND d2.IRLikedPartner IS NOT NULL
RETURN a.Part_num                                      AS rater_a,
       b.Part_num                                      AS rater_b,
       d1.IRLikedPartner                               AS a_liked_b,
       d2.IRLikedPartner                               AS b_liked_a,
       abs(d1.IRLikedPartner - d2.IRLikedPartner)      AS asymmetry
ORDER BY asymmetry DESC;
```

In a rectangular format, the same computation requires a self-join on the original table, filtering by mutmatch = 1 on both sides and using two different aliases for the same variable. The graph query traverses existing structure; the SQL query reconstructs structure that was never represented. This distinction, modest in computational terms, is consequential in epistemological ones: the graph makes the researcher encounter the asymmetry as a feature of the data model, not as a derived statistic.

5.2 Structural Missingness and the Preservation of Absent Entities

Rectangular data formats handle missing observations in two ways: by inserting placeholder values (typically NA or 0) or by omitting rows entirely. Both approaches conflate structurally distinct situations. A participant who attended the speed-dating event but did not match with anyone is structurally different from a participant who matched but did not respond to follow-up surveys, which is in turn structurally different from a participant who responded to some follow-up surveys but not others. In the CSV file used by Selterman et al. (2023), all three situations produce the same NA encoding in the FURomInterest column.

The graph handles these situations by representing absence as a structural feature rather than a numerical encoding. A participant without a mutual match simply lacks any outgoing `FOLLOWED_UP` edges from their `DateRating` nodes. A participant who dropped out after week 3 has `FollowUp` nodes for weeks 1 through 3 and no nodes for weeks 4 through 10. The absence of a node is not an NA; it is the absence of an entity that did not exist in the study. This distinction becomes analytically meaningful when modeling longitudinal attrition. In the Speed-Dating dataset, 1,152 `FollowUp` nodes were created from 28,160 potential rater-partner-week combinations — a 95.9% structural absence rate. In a multilevel model estimated from the rectangular file, this attrition is handled implicitly by the full-information maximum likelihood estimator, which assumes that missingness is a function of observed variables. In the graph, the researcher is confronted directly with the network structure of attrition: which dyads dropped out early, which persisted across all ten weeks, and whether dropout is clustered within particular sessions or participants. These questions are answerable with graph traversal queries before any statistical model is specified.

5.3 Endogenous Network Structure as a Source of Unmodeled Dependence

The most consequential methodological implication of the graph representation concerns a limitation of the original analysis by Selterman et al. (2015) that becomes structurally visible only when the data are modeled as a graph. The speed-dating design generates at least three sources of statistical dependence among observations: a rater effect (some participants rate all their partners more generously or more harshly than average), a partner popularity effect (some partners receive consistently higher ratings from multiple independent raters), and a dyadic reciprocity effect (mutual-match pairs show correlated ratings). Selterman et al. (2015) modeled the rater effect explicitly through random intercepts at the participant level, reporting intraclass correlations between .14 and .49 for Level 2 variance in their two-level models. However, the partner popularity effect and the dyadic reciprocity effect were not modeled, and in the rectangular format, neither is immediately visible as a structural feature of the data.

In the labeled property graph, all three sources of dependence are structurally explicit and visually evident after some cypher queries such as the one that produced Figure 3. The rater effect corresponds to the out-degree of each Participant node via

HAD_DATE edges. The partner popularity effect corresponds to the in-degree of each Participant node via `WITH` edges — the number of distinct raters who evaluated a given partner. The dyadic reciprocity effect corresponds to closed two-paths between mutual-match `DateRating` nodes. None of these requires auxiliary computation; all three are properties of the graph that can be queried directly. The following query makes the partner popularity structure explicit:

```
// Query B: Partner popularity | in-degree and mean received rating
MATCH (d:DateRating)-[:WITH]->(partner:Participant)
WITH  partner,
      count(d)              AS n_raters,
      avg(d.IRLikedPartner) AS mean_received_rating,
      stdev(d.IRLikedPartner) AS sd_received_rating
RETURN partner.Part_num    AS partner_id,
       n_raters,
       round(mean_received_rating, 2) AS mean_received_rating,
       round(sd_received_rating,  2)  AS sd_received_rating
ORDER BY mean_received_rating DESC
LIMIT 20;
```

Applying this query to the Speed-Dating dataset reveals substantial variability in partner popularity. Partners were evaluated by between 1 and 20 raters (M = 7.45, SD = 3.35), and the standard deviation of mean received ratings across partners was 1.18 points on the 9-point IRLikedPartner scale — approximately 41% of the total variance in that variable. This figure is comparable in magnitude to the Level 2 ICC reported by (Selterman et al., 2015) for the rater effect, suggesting that the partner popularity effect is of similar size to the rater effect that was modeled. In a model that omits random effects for the partner, this variance is absorbed into the residual, potentially attenuating standard errors and inflating the apparent precision of fixed-effect estimates. The appropriate statistical model for this design is a crossed random effects model in which both the rater and the partner contribute independent sources of variance (Kenny, Kashy, & Cook, 2006). In R, this specification takes the form:

```
# Crossed random effects model
# Level 1: weekly observations
# Random effect 1: rater (participant)
# Random effect 2: partner (MatchID_clean) | absent in Selterman et al.

model_crossed <- lmer(
  romantic_interest ~
    initial_attraction_c +
    week_c +
    sex +
    initial_attraction_c:week_c +
    (1 | participant) +
    (1 | partner),
  data    = dating_graph,
  REML    = TRUE,
```

```
  control = lmerControl(optimizer = 'bobyqa')
)
```

Note that the partner random effect requires that MatchID_clean be exported from the graph alongside Part_num in Query B. Both identifiers are properties of the DateRating node and are available in the same traversal that retrieves IRLikedPartner and FURomInterest.

It is important to be precise about what this observation implies for the conclusions of Selterman et al. (2015). Their primary finding that the association between partner characteristics and romantic interest was not moderated by participant gender is unlikely to be reversed by the addition of partner random effects, because the gender moderation test operates at a level of the model that is largely orthogonal to the random effect structure. What the omission of partner random effects does affect is the precision of all fixed-effect estimates and the correct partitioning of variance across levels. These are not trivial concerns for a paper published in a replication context, where the accurate estimation of effect sizes is central to the inferential claims.

The broader methodological point for the present tutorial is as follows. The graph does not solve the modeling challenges posed by speed-dating designs, but it makes them structurally impossible to overlook. A researcher who represents speed-dating data as a labeled property graph encounters the partner popularity structure as an explicit feature of the schema — the in-degree of each Participant node is a property that any graph query will surface. A researcher who works exclusively with the rectangular file can, and in at least one prominent replication did, proceed to multilevel modeling without ever confronting the partner popularity effect as a structural feature of the data. The epistemological contribution of the graph representation is not computational but representational: it shifts the burden of justification from the decision to model partner effects to the decision to omit them.

5.4 Schema Flexibility and Theoretical Evolution

A recurring challenge in longitudinal psychological research is that theoretical models evolve during data collection, requiring modifications to the data structure that are costly in relational databases. Adding a new construct, revising the operationalization of an existing variable, or connecting data from a new source typically requires altering table schemas, updating foreign key constraints, and migrating existing data. These operations are not free of risk because they can introduce inconsistencies and might require careful documentation to remain reproducible for other researchers.

In a labeled property graph, the schema is inherently additive. New node types, relationship types, and properties can be introduced without modifying the existing graph structure or invalidating existing queries. The Speed-Dating graph illustrates this property concretely. The current schema includes three node types and four relationship types. If a researcher wished to add session-level information — for example, distinguishing the Fall 2012 from the Spring 2013 cohort, or linking participants to the specific speed-dating event they attended — this would require adding a fourth node type (e.g., `Session`) and two new relationship types (`ATTENDED` and `PART_OF`) without altering any existing node or relationship. Existing queries about participants, date ratings, and follow-ups would continue to execute correctly on the modified graph.

This flexibility is particularly relevant for psychological research because it aligns with how theoretical models actually develop: not by replacing prior representations, but by adding new constructs and connections to an existing framework. From a methodological perspective, graph database schema evolution is not an ad-hoc adjustment to the data structure but a representation of genuine scientific progress — a point that connects to broader arguments about the role of ontological commitments in psychological methodology (Burgos, 2025).

6 Conclusions and limitations

References

Bak, M.Y.S., Plavnick, J.B., Dueñas, A.D., Brodhead, M.T., Avendaño, S.M., Wawrzonek, A.J., ... Oteto, N. (2021). The use of automated data collection in applied behavior analytic research: A systematic review. *Behavior Analysis: Research and Practice*, *21*(4), 376, https://doi.org/10.1037/bar0000228

Barrasa, J., & Webber, J. (2023). *Building knowledge graphs: A practitioner's guide.* Sebastopol, CA: O'Reilly.

Borsboom, D., & Cramer, A. (2013, 03). Network analysis: An integrative approach to the structure of psychopathology. *Annual Review of Clinical Psychology*, *9*(Volume 9, 2013), 91–121, https://doi.org/10.1146/annurev-clinpsy-050212-185608

Burgos, J. (2025). Getting ontologically serious about the replication crisis in psychology. *Journal of Theoretical and Philosophical Psychology*, *45*(2), 79–100, https://doi.org/10.1037/teo0000281

Cronbach, L.J., & Meehl, P.E. (1955). Construct validity in psychological tests. *Psychological Bulletin*, *52*(4), 281–302, https://doi.org/10.1037/h0040957

Elcoro, M., Diller, J.W., Correa, J.C. (2023). Promoting reciprocal relations across subfields of behavior analysis via collaborations. *Perspective on Behavior Science*, *46*, 431-446, https://doi.org/10.1007/s40614-023-00386-x

Gonzalez, M.C., Hidalgo, C.A., Barabasi, A.-L. (2008). Understanding individual human mobility patterns. *Nature*, *453*(7196), 779–782, https://doi.org/10.1038/nature06958

Guastello, S.J., Koopmans, M., Pincus, D. (2009). *Chaos and complexity in psychology: The theory of nonlinear dynamical systems*. Cambridge University Press.

Guimerà, R., Uzzi, B., Spiro, J., Amaral, L. (2005, 04). Team assembly mechanisms determine collaboration network structure and team performance. *Science*, *308*(5722), 697–702, https://doi.org/10.1126/science.1106340

Kenny, D.A., Kashy, D.A., Cook, W.L. (2006). *Dyadic data analysis*. New York: The Guilford Press.

Krakauer, D. (2019). *Worlds hidden in plain sight: The evolving idea of complexity at the Santa Fe Institute*. New Mexico: The Santa Fe Institute Press.

Luce, R.D. (1999). Where is mathematical modeling in psychology headed? *Theory & Psychology*, *9*(6), 723–737,

Luke, D.A. (2015). *A User's Guide to Network Analysis in R*. Cham: Springer.

Mair, P. (2018). *Modern Psychometrics with R*. Cham: Springer.

R Core Team (2025). R: A language and environment for statistical computing [Computer software manual]. Vienna, Austria. Retrieved from https://www.R-project.org/

Robinson, I., Webber, J., Eifrem, E. (2015). *Graph databases: New opportunities for connected data*. Sebastopol, California: O'Reilly.

Selterman, D.F., Chagnon, E., Mackinnon, S.P. (2015). Do Men and Women Exhibit Different Preferences for Mates? A Replication of Eastwick and Finkel (2008). *SAGE Open*, *5*(3), 1-14, https://doi.org/10.1177/2158244015605160

Selterman, D.F., Chagnon, E., Mackinnon, S.P. (2023, Sep). *Replication of Eastwick and Finkel (2008, JPSP)*. Open Science Framework. Retrieved from https://osf.io/ng6cc/overview

Soto, P.L. (2025). Relational databases for behavior science. *Perspectives on Behavior Science*, *48*, 909-929, https://doi.org/10.1007/s40614-025-00486-w

Van Bruggen, R. (2014). *Learning neo4j*. Birmingham, UK: Packt Publishing.

Vezzoli, M., & Zogmaister, C. (2023). An introductory guide for conducting psychological research with big data. *Psychological Methods*, *28*(3), 580–599, https://doi.org/10.1037/met0000513

Wickham, H., Miller, E., Smith, D. (2025). haven: Import and Export 'SPSS', 'Stata' and 'SAS' Files [Computer software manual]. Retrieved from https://CRAN.R-project.org/package=haven (R package version 2.5.5)

Wickham, H., Çetinkaya Rundel, M., Grolemund, G. (2023). *R for Data Science* (2nd ed.). Sebastopol, CA: O'Reilly.